

06. 순환 신경망 (RNN)

순서가 있는 데이터를 다루는 방법. 기본 RNN에서 시작해 LSTM, 주식 예측, 감성 분석, 하이퍼파라미터 최적화, 이상탐지까지 이어진다.

1. 왜 RNN인가

일반 신경망은 입력을 한 덩어리로 받는다. [어제, 오늘, 내일] 을 넣어도 [내일, 어제, 오늘] 을 넣어도 같은 취급이다. **순서 정보가 없다.**

RNN은 이전 시점의 계산 결과를 다음 시점으로 넘긴다. 그래서 "앞에 무엇이 있었는지"가 뒤쪽 계산에 반영된다.

구조	입력 처리	순서 정보
MLP	전부 한 번에	없음
CNN	지역 패턴	공간적 인접성만
RNN	한 시점씩 순차	유지

2. nn.RNN / nn.LSTM 입출력

```
self.rnn = nn.RNN(input_size, hidden_size,
                  num_layers = 1, batch_first = True)

out, hidden = self.rnn(x)
```

batch_first = True 를 쓰면 텐서 모양이 (배치, 시퀀스 길이, 특징 수) 가 된다. 안 쓰면 (시퀀스, 배치, 특징) 이라 헷갈리니 항상 붙이는 편이 낫다.

반환값 두 개의 차이

변수	담긴 것	shape
out	모든 시점의 은닉 상태	(B, L, H)
hidden	마지막 시점의 은닉 상태	(층수, B, H)

분류처럼 문장 전체를 하나로 요약해야 하면 hidden[-1] 을 쓴다. 시점마다 출력이 필요하면 out 을 쓴다.

hidden[-1] 의 -1 은 **마지막 층**이라는 뜻이다. 시점이 아니다. 자주 헷갈리는 지점.

LSTM은 상태가 두 개

```
out, (hidden, cell) = self.lstm(x)
```

cell 이 장기 기억을 담는다. 기본 RNN은 시점이 길어지면 앞쪽 정보가 사라지는데(기울기 소실), LSTM은 게이트로 무엇을 지우고 무엇을 남길지 조절해서 이를 완화한다.

게이트	역할
forget	이전 기억 중 버릴 것 결정
input	새 정보 중 저장할 것 결정
output	기억 중 지금 내보낼 것 결정

3. 가변 길이 시퀀스 다루기

문장 길이가 제각각이면 배치로 묶을 수 없다. 패딩으로 길이를 맞춘 뒤, 패딩 자리를 계산에서 빼야 한다.

```
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence

x_train = pad_sequence(seqs, batch_first = True, padding_value = 0)
packed = pack_padded_sequence(embedded, lengths,
                              batch_first = True, enforce_sorted = False)
```

- pad_sequence — 가장 긴 시퀀스에 맞춰 0으로 채운다
- pack_padded_sequence — 실제 길이만큼만 RNN이 돌도록 묶는다
- enforce_sorted = False — 기본값이 True 라 길이 내림차순 정렬을 요구한다. 안 붙이면 에러

`nn.Embedding(..., padding_idx = 0)` 을 함께 쓰면 패딩 토큰의 임베딩이 0으로 고정되고 학습되지도 않는다.

길이 계산 주의 — 길이는 반드시 **변환이 끝난 결과**에서 재야 한다. 토큰화 직후 길이와 인덱스 변환 후 길이가 다르면(OOV 제거 등) `pack_padded_sequence` 에서 에러가 나거나 뒷부분이 조용히 잘린다.

4. 시계열 윈도우

시계열은 "과거 N개로 다음 값을 맞힌다"는 형태로 바뀌어야 학습할 수 있다.

```
def create_sequence(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)
```

슬라이딩 윈도우를 한 칸씩 밀면서 입력과 정답 쌍을 만든다.

분할 원칙 — 시계열은 **절대 무작위로 섞지 않는다**. 미래 데이터로 과거를 맞추는 셈이 되어 성능이 부풀려진다.

```
python train_size = int(len(X) * 0.8) X_train, y_train = X[:train_size], y[:train_size] X_test, y_test = X[train_size:], y[train_size:]
```

`DataLoader` 의 `shuffle` 도 train에만 `True` , test에는 `False` 다.

5. 주식 가격 예측

```
ticker = "005930.KS"          # 삼성전자
df = yf.download(ticker, start = ..., end = ...)
df = df.dropna()
scaler = MinMaxScaler()
```

정규화가 필수인 이유 — 주가는 5만~8만 같은 큰 값이다. 그대로 넣으면 초기 손실이 거대해지고 기울기가 폭주한다. `MinMaxScaler` 로 0~1 범위에 넣고, 예측 후 `inverse_transform` 으로 되돌린다.

스케일러는 **훈련 데이터로만 fit** 해야 한다. 전체로 fit하면 테스트 구간의 최댓값 정보가 새어 들어간다.

6. 감성 분석과 어텐션

어휘 사전 만들기

```
def build_vocab(tokenized_texts, min_freq = 1):
    counter = Counter(w for text in tokenized_texts for w in text)
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, freq in counter.items():
        if freq >= min_freq:
            vocab[word] = len(vocab)
    return vocab
```

`<pad>` 를 0번에 두는 게 관례다. `padding_idx = 0` 과 짝이 맞는다. `<unk>` 는 사전에 없는 단어를 받아내는 자리다.

어텐션 층

```
class AttentionLayer(nn.Module):
    def forward(self, x, mask = None):
        attention = self.context(torch.tanh(self.attention(x))).squeeze(-1)
        if mask is not None:
            attention = attention.masked_fill(mask == 0, -1e10)
        attention_weights = F.softmax(attention, dim = 1)
        weighted_output = torch.bmm(attention_weights.unsqueeze(1), x).squeeze(1)
        return weighted_output, attention_weights
```

핵심은 **마스킹**이다. 패딩 자리에 아주 작은 음수($-1e10$)를 넣으면 softmax를 통과하면서 가중치가 0에 수렴한다. 의미 없는 자리에 주목하지 않게 만드는 장치다.

```
mask = (text != 0)      # 패딩(0)이 아닌 위치만 True
```

어텐션 가중치를 꺼내 보면 모델이 어느 단어를 보고 판단했는지 확인할 수 있다. 이게 설명가능성(XAI)의 출발점이다.

7. 하이퍼파라미터 최적화

탐색 공간은 금방 폭발한다

11개 항목에 각각 3~5개 후보를 두면 조합이 **184만 개**가 된다. 1회 학습에 10초만 잡아도 213일이다.

$$4 \times 4 \times 5 \times 2 \times 5 \times 4 \times 4 \times 4 \times 4 \times 3 \times 3 = 1,843,200$$

전수 탐색(Grid Search)은 현실적으로 불가능하다. 랜덤 샘플링이나 베이지안 최적화가 전제다.

베이지안 최적화

```
from skopt import gp_minimize
from skopt.space import Real, Integer, Categorical

space = [Integer(64, 512, name = "hidden_size"),
         Real(1e-5, 1e-2, prior = "log-uniform", name = "learning_rate"),
         Categorical([16, 32, 64, 128], name = "batch_size")]
```

prior = "log-uniform" 이 학습률에서 중요하다. 학습률은 선형이 아니라 **배수로** 움직이기 때문이다. **uniform** 으로 두면 표본의 90%가 **1e-3** 이상에 몰려 작은 학습률 영역을 사실상 탐색하지 못한다.

부호 규칙 — gp_minimize 는 **최소화만** 한다.

목적 함수가 반환하는 것	반환 방식
손실 (작을수록 좋음)	return loss
정확도 · F1 · AUC (클수록 좋음)	return -accuracy

부호 뒤집기는 **한 번만** 한다. 두 번 뒤집으면 상쇄되어 원래 값이 되고, 최적화기가 반대 방향으로 간다.

최고점 갱신 방향

초기값과 비교 연산자는 짝을 맞춰야 한다.

목적	초기값	비교
손실 최소화	float("inf")	score < best_score
정확도 최대화	float("-inf")	score > best_score

float("inf") 로 시작해놓고 score > best_score 로 비교하면 무한대보다 큰 값이 없으니 **갱신이 한 번도 일어나지 않는다**. 에러 없이 None 만 반환된다.

8. 학습률 스케줄러

```
class AdaptiveLearningRateScheduler:
    def step(self, current_score):
        self.current_epoch += 1
        if self.current_epoch <= self.warmup_epochs:
            lr_scale = self.current_epoch / self.warmup_epochs
            for g in self.optimizer.param_groups:
                g["lr"] = g["initial_lr"] * lr_scale
            return
        ...
```

워밍업은 학습 초반에 학습률을 0에서 목표치까지 서서히 올리는 기법이다. 무작위 초기 가중치에 큰 학습률을 바로 때리면 발산하기 쉬워서다.

param_group["initial_lr"] 은 파이토치 옵티마이저에 **기본으로 없는 키**다. 직접 만든 스케줄러라면 미리 넣어둬야 한다.

```
for g in optimizer.param_groups:
    g["initial_lr"] = g["lr"]
```

표준 스케줄러 비교

스케줄러	동작	쓰는 상황
StepLR(step_size=30, gamma=0.1)	30에포크마다 $\times 0.1$	일정을 미리 알 때
ReduceLROnPlateau(patience=10)	정체되면 낮춤	가장 무난한 선택
CosineAnnealingLR(T_max=200)	코사인 곡선으로 감소	전체 에포크 고정 시

ReduceLROnPlateau 는 scheduler.step(val_loss) 처럼 검증 손실을 넘기고, 배치가 아니라 에포크 끝에서 호출한다.

9. LSTM 오토인코더 이상탐지

정상 데이터만으로 학습시켜 "복원이 잘 안 되는 구간 = 이상"으로 판정하는 방식이다. 라벨이 거의 없는 상황에서 강력하다.

```
class LSTMAutoencoder(nn.Module):
    def forward(self, x):
        _, (hidden, cell) = self.encoder(x)
        decoder_input = hidden.permute(1, 0, 2).repeat(1, self.sequence_length, 1)
        output, _ = self.decoder(decoder_input)
        return self.fc(output)
```

인코더가 시퀀스를 벡터 하나로 압축하고, 그 벡터를 시퀀스 길이만큼 복제해 디코더가 원본을 복원한다.

임계값 설정

```
def set_threshold(errors, z_score = 3.0):
    return np.mean(errors) + z_score * np.std(errors)
```

이 방식에는 함정이 있다. 이상치의 큰 오차가 표준편차 자체를 키워서, $\text{mean} + 3\sigma$ 가 최대 오차보다 높아지는 경우가 생긴다. 그러면 아무것도 탐지되지 않는다.

z_score 를 3.0 $\rightarrow$ 2.0 $\rightarrow$ 1.5 로 낮추면서 탐지 개수가 적절한 지점을 찾거나, 백분위수 방식(np.percentile(errors, 99))으로 바꾼다.

검수에서 찾은 동작 오류

수업 중 실제로 걸렸던 것들이다. 전부 문법 오류가 아니라 조용히 잘못되거나 실행 시점에 터지는 종류다.

1. 이상탐지가 전혀 안 됨 — trend 계수

```
trend = 0.05 * np.arange(n_samples)    # 잘못
trend = 0.005 * np.arange(n_samples)    # 정상
```

추세 계수가 10배면 y축 범위가 6에서 50으로 커진다. 사인파와 이상치의 절대 크기는 그대로라, 화면상 변동폭이 8분의 1로 눌러 직선처럼 보인다.

	trend 0.005	trend 0.05
사인 진폭	2.00	2.00 (동일)
전체 y 범위	6.33	49.47
사인 변동 / 전체범위	31.6%	4.0%
정규화 후 이상치 틱	0.2154	0.0280

정규화 후 이상치가 0.028까지 작아지면 오토인코더의 재구성 오차 바닥(약 0.02~0.05)에 묻혀 탐지가 불가능해진다.

2. 차원 오류 — squeeze vs unsqueeze

```
sequence_tensor = torch.FloatTensor(seq).squeeze(0).unsqueeze(2)    # 잘못
sequence_tensor = torch.FloatTensor(seq).unsqueeze(0).unsqueeze(2)    # 정상
```

squeeze(0) 은 크기가 1인 축만 없앤다. 0번 축이 20이면 아무 일도 하지 않고 1차원인 채로 남는다. 그 상태에서 unsqueeze(2) 를 부르면 에러가 난다.

```
IndexError: Dimension out of range (expected to be in range of [-2, 1], but got 2)
```

메시지의 [-2, 1] 이 "이 텐서는 1차원"이라는 신호다.

3. 그래프 좌표계 — textcoords 대소문자

```
textcoords = "Offset Points"    # 잘못
textcoords = "offset points"    # 정상
```

matplotlib의 좌표계 이름은 전부 소문자다. 그리고 annotate 호출 시점에는 검증하지 않고 레이아웃 계산 시점에 터지기 때문에, 에러가 plt.tight_layout() 을 가리킨다. 원인 위치와 에러 위치가 다르다.

4. 학습이 안 됨 — optimizer.step() 누락

```
loss.backward()
optimizer.step()    # 이 줄이 빠지면 기울기만 구하고 가중치는 그대로
```

에러가 안 나서 학습이 되는 줄 안다. 50에포크를 돌려도 손실이 거의 안 줄면 이걸 의심한다.

5. 탐색 공간 전체 생성

```
param_combinations = list(ParameterGrid(param_grid))    # 184만 개를 먼저 만들
if len(param_combinations) > max_trials:
    param_combinations = random.sample(param_combinations, max_trials)
```

50개만 뽑을 건데 184만 개를 전부 메모리에 만든 뒤 샘플링한다. ParameterSampler 를 쓰면 필요한 개수만 생성된다.

6. num_layers = 1 에 dropout > 0

파이토치 RNN 계열의 dropout 은 층과 층 사이에만 걸린다. 1층이면 적용될 자리가 없어 경고가 뜨고, 그 조합들은 dropout 값이 무의미하게 중복 실행된다. 탐색할 때 num_layers == 1 이면 dropout 을 0으로 고정하는 식으로 걸러두는 편이 낫다.

정리 포인트

- out 은 모든 시점, hidden 은 마지막 시점. hidden[-1] 의 -1 은 층 인덱스
- 가변 길이는 pad_sequence + pack_padded_sequence(enforce_sorted = False)
- 길이는 변환 후 결과에서 재기
- 시계열은 무작위 분할 금지, 스케일러는 훈련 데이터로만 fit
- 어텐션 마스킹은 masked_fill(mask == 0, -1e10)
- gp_minimize 는 최소화 전용 — 부호 뒤집기는 한 번만
- 초기값 inf 와 비교 < 는 세트, -inf 와 > 가 세트
- 워밍업에는 initial_lr 을 직접 심어줘야 함
- mean + 3σ 임계값은 이상치가 σ를 키워 스스로 무력화될 수 있음